ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

MÔN: TOÁN ỨNG DỤNG VÀ THỐNG KÊ

ĐỒ ÁN 1: Ma Trận và Cơ Sở của Tính Toán Khoa Học

Sinh viên thực hiện:
Nguyễn Khánh Đăng (24120171)
Đặng Quang Huy (24120322)
Trương Bảo Nguyên (24120397)
Võ Gia Phúc (24120413)
Trần Nguyễn Lâm Gia Thụ (24120458)

Giáo viên hướng dẫn:
ThS. Võ Nam Thục Đoan
ThS. Lê Nhựt Nam

Tp. Hồ Chí Minh, tháng 3 năm 2026

Mục lục

1 GIỚI THIỆU CHUNG **2**

2 PART 1: CÁC PHÉP TOÁN MA TRẬN CƠ BẢN **2**
2.1 Khử Gauss và Thế ngược (gaussian.py) . . . 2
2.1.1 Cơ sở lý thuyết . . . 2
2.1.2 Cài đặt thuật toán . . . 2
2.2 Tính định thức (determinant.py) . . . 2
2.2.1 Cơ sở lý thuyết . . . 2
2.2.2 Cài đặt thuật toán . . . 3
2.3 Ma trận nghịch đảo (inverse.py) . . . 3
2.3.1 Cơ sở lý thuyết . . . 3
2.3.2 Cài đặt thuật toán . . . 3
2.4 Hạng và Các không gian cơ sở (rank_basis.py) . . . 4
2.4.1 Cơ sở lý thuyết . . . 4
2.4.2 Cài đặt thuật toán . . . 4

3 PART 2: PHÂN RÃ MA TRẬN VÀ TRỰC QUAN HÓA **5**
3.1 Chéo hóa ma trận (diagonalization.py) . . . 5
3.1.1 Cơ sở lý thuyết . . . 5
3.1.2 Cài đặt thuật toán . . . 5
3.2 Phân rã giá trị kỳ dị SVD (decomposition.py) . . . 5
3.2.1 Cơ sở lý thuyết . . . 5
3.2.2 Quy trình cài đặt . . . 5
3.3 Trực quan hóa biến đổi tuyến tính với Manim (manim_scene.py) . . . 6
3.3.1 Ma trận thử nghiệm và ý nghĩa hình học . . . 6
3.3.2 Cấu trúc kịch bản Animation . . . 6
3.3.3 Kết quả thực nghiệm và Hiệu ứng . . . 6
3.3.4 Ứng dụng thực tiễn và Thách thức kỹ thuật . . . 7

4 PART 3: GIẢI HỆ PHƯƠNG TRÌNH TUYẾN TÍNH VÀ ĐÁNH GIÁ HIỆU NĂNG **7**
4.1 Mục tiêu và Đặt vấn đề . . . 7
4.2 Cài đặt thuật toán cốt lõi (solvers.py) . . . 7
4.2.1 Khử Gauss với Partial Pivoting (`solve_gauss`) . . . 7
4.2.2 Phân rã ma trận SVD (`solve_svd`) . . . 8
4.2.3 Phương pháp lặp Gauss-Seidel (`gauss_seidel`) . . . 8
4.3 Quy trình thực nghiệm (benchmark.py) . . . 9
4.4 Phân tích kết quả thực nghiệm . . . 9
4.4.1 Hiệu năng và Độ phức tạp thời gian (Time Complexity) . . . 9
4.4.2 Tính ổn định số và Số điều kiện (Numerical Stability) . . . 10
4.5 Tổng kết và Kiến nghị . . . 10

5 KẾT LUẬN **10**

6 TÀI LIỆU THAM KHẢO **10**

7 BẢNG PHÂN CÔNG CÔNG VIỆC **11**

1 GIỚI THIỆU CHUNG

Đồ án này tập trung vào việc cài đặt các thuật toán nền tảng của Đại số tuyến tính bằng ngôn ngữ Python, từ các phép biến đổi ma trận cơ bản đến các ứng dụng phân rã và trực quan hóa dữ liệu.

2 PART 1: CÁC PHÉP TOÁN MA TRẬN CƠ BẢN

2.1 Khử Gauss và Thế ngược (gaussian.py)

2.1.1 Cơ sở lý thuyết

Để giải hệ phương trình $Ax = b$, thuật toán thực hiện:

1. **Chọn phần tử trội (Partial Pivoting):** Tìm hàng có giá trị tuyệt đối lớn nhất tại cột đang xét để hạn chế sai số dấu phẩy động.
2. **Biến đổi sơ cấp:** Đưa ma trận bổ sung $[A|b]$ về dạng bậc thang U.
3. **Thế ngược:** Giải hệ từ hàng cuối cùng lên để tìm nghiệm duy nhất hoặc hệ nghiệm (nếu có ẩn tự do).

2.1.2 Cài đặt thuật toán

```python
def gaussian_eliminate(A, b):
    n, m = len(A), len(A[0])
    M = [[float(x) for x in A[i]] + [float(b[i])] for i in range(n)]
    s = 0  # So lan doi hang
    epsilon = 1e-12

    for k in range(min(n, m)):
        p = k
        for i in range(k + 1, n):
            if abs(M[i][k]) > abs(M[p][k]): p = i
        if abs(M[p][k]) < epsilon: continue
        if p != k:
            M[k], M[p] = M[p], M[k]
            s += 1
        for i in range(k + 1, n):
            factor = M[i][k] / M[k][k]
            for j in range(k, m + 1):
                M[i][j] -= factor * M[k][j]
    # Tra ve ma tran bac thang, ket qua substitution va so lan doi hang
    return M, back_substitution(M), s
```

Listing 1: Mã nguồn giải hệ phương trình bằng khử Gauss

2.2 Tính định thức (determinant.py)

2.2.1 Cơ sở lý thuyết

Định thức được tính dựa trên tính chất của ma trận tam giác. Sau khi khử Gauss đưa ma trận vuông về dạng bậc thang U, định thức được tính bằng tích các phần tử đường chéo chính nhân với hệ số $(-1)^s$:

$$\det(A) = (-1)^s \times \prod_{i=0}^{n-1} M[i][i] \tag{1}$$

2.2.2 Cài đặt thuật toán

```
def determinant(A):
    n = len(A)
    for row in A:
        if len(row) != n: return "Ma tran khong vuong"

    b = [0.0] * n
    M, _, s = gaussian_eliminate(A, b)

    det_val = (-1.0) ** s
    for i in range(n):
        det_val *= M[i][i]
    return det_val
```

Listing 2: Mã nguồn tính định thức

2.3 Ma trận nghịch đảo (inverse.py)

2.3.1 Cơ sở lý thuyết

Để tìm ma trận nghịch đảo A^{-1} của ma trận vuông A, nhóm triển khai phương pháp **Gauss-Jordan** trên ma trận bổ sung $(A|I)$.

- Khởi tạo ma trận bổ sung kích thước $n \times 2n$ bằng cách ghép ma trận đơn vị I vào bên phải ma trận A.
- Thực hiện các phép biến đổi dòng sơ cấp để đưa khối bên trái về ma trận đơn vị I.
- Khi đó, khối bên phải chính là ma trận nghịch đảo A^{-1} cần tìm.

Trong quá trình này, kỹ thuật *Partial Pivoting* kết hợp với kiểu dữ liệu `Fraction` giúp đảm bảo thuật toán không bị dừng đột ngột do sai số làm tròn và phát hiện chính xác các ma trận không khả nghịch ($\det(A) = 0$).

2.3.2 Cài đặt thuật toán

```
def inverse(matrix: list) -> list:
    n = len(matrix)
    if any(len(row) != n for row in matrix):
        raise ValueError("Ma tran dau vao buoc phai la ma tran vuong")

    # Tao ma tran bo sung augmented [A|I] dung Fraction de bao toan do
    chinh xac
    augmented = []
    for c in range(n):
        row = [Fraction(x) for x in matrix[c]]
        identity_row = [Fraction(1) if c == j else Fraction(0) for j in
    range(n)]
        augmented.append(row + identity_row)

    for c in range(n):
        pivot_row, value = get_partial_pivot(augmented, c, c)
        if value == 0:
            raise ValueError("Ma tran co dinh thuc = 0 nen khong the
    nghich dao")

```

```
        if pivot_row != c:
            augmented[c], augmented[pivot_row] = augmented[pivot_row],
    augmented[c]

        pivot_value = augmented[c][c]
        augmented[c] = [x / pivot_value for x in augmented[c]]

        for k in range(n):
            if k != c:
                factor = augmented[k][c]
                for j in range(c, 2 * n):
                    augmented[k][j] -= factor * augmented[c][j]

    return [row[n:] for row in augmented]
```

Listing 3: Mã nguồn tìm ma trận nghịch đảo bằng Gauss-Jordan

2.4 Hạng và Các không gian cơ sở (rank_basis.py)

2.4.1 Cơ sở lý thuyết

Hệ thống xác định các đặc trưng của ma trận thông qua việc biến đổi về dạng bậc thang rút gọn (Row Echelon Form - RREF). Các thông tin được trích xuất bao gồm:

- **Hạng (Rank):** Số lượng các cột chứa phần tử chốt (pivot) trong RREF.
- **Cơ sở không gian cột (Column Space):** Tập hợp các cột tại vị trí tương ứng với cột pivot, trích xuất từ ma trận gốc.
- **Cơ sở không gian dòng (Row Space):** Tập hợp các dòng khác không trong ma trận RREF.
- **Cơ sở không gian nghiệm (Null Space):** Giải hệ phương trình thuần nhất $Ax = 0$ bằng cách xác định các biến tự do (cột không có pivot) và tính toán các biến pivot tương ứng.

2.4.2 Cài đặt thuật toán

```
def rank_and_basis(matrix: list) -> tuple:
    rows, cols = len(matrix), len(matrix[0])
    rref_matrix, pivot_cols = get_rref(matrix)

    # Hang ma tran
    rank = len(pivot_cols)

    # Co so khong gian cot (lay tu ma tran goc)
    col_basis = [[matrix[r][c] for r in range(rows)] for c in pivot_cols]

    # Co so khong gian dong (lay tu cac dong khac 0 trong RREF)
    row_basis = [rref_matrix[r] for r in range(rank)]

    # Co so khong gian nghiem (Null space)
    free_vars = [j for j in range(cols) if j not in pivot_cols]
    null_basis = []
    for f_var in free_vars:
        vec = [Fraction(0)] * cols
        vec[f_var] = Fraction(1)
        for i, p_col in enumerate(pivot_cols):
            vec[p_col] = -rref_matrix[i][f_var]
        null_basis.append(vec)
```

```

    return rank, col_basis, row_basis, null_basis
```

Listing 4: Mã nguồn trích xuất hạng và cơ sở không gian

3 PART 2: PHÂN RÃ MA TRẬN VÀ TRỰC QUAN HÓA

3.1 Chéo hóa ma trận (diagonalization.py)

3.1.1 Cơ sở lý thuyết

Ma trận $\mathbf{A} \in \mathbb{R}^{n\times n}$ chéo hóa được nếu tồn tại ma trận $\mathbf{P}$ khả nghịch và ma trận đường chéo $\mathbf{D}$ sao cho:

$$\mathbf{A} = \mathbf{P}\mathbf{D}\mathbf{P}^{-1} \tag{2}$$

Trong đó, đường chéo của $\mathbf{D}$ chứa các giá trị riêng, và các cột của $\mathbf{P}$ là các vector riêng tương ứng. Nhóm sử dụng thuật toán **Faddeev–LeVerrier** để tìm đa thức đặc trưng thay vì tính định thức trực tiếp, giúp tối ưu hóa việc cài đặt mã nguồn.

3.1.2 Cài đặt thuật toán

Nhóm triển khai phương pháp hybrid để tìm nghiệm đa thức đặc trưng:

- Bậc 2 và 3: Dùng công thức đại số (Cardano/Vieta) để đảm bảo độ chính xác tuyệt đối.
- Bậc 4: Dùng phương pháp lặp QR (QR Iteration) với phân rã Gram-Schmidt.

```
def faddeev_leverrier(A):
    n = len(A)
    M = [[0.0]*n for _ in range(n)]
    coeffs = []
    for k in range(1, n + 1):
        Ak = [row[:] for row in A] if k == 1 else _mat_mul(A, M)
        c = -(1.0 / k) * _trace(Ak)
        coeffs.append(c)
        M = _mat_add(Ak, _mat_scale(_identity(n), c))
    return coeffs[::-1]
```

Listing 5: Thuật toán Faddeev–LeVerrier tìm hệ số đa thức đặc trưng

3.2 Phân rã giá trị kỳ dị SVD (decomposition.py)

3.2.1 Cơ sở lý thuyết

Phân rã SVD biến đổi ma trận $\mathbf{A}_{m\times n}$ thành tích của ba ma trận: $\mathbf{A} = \mathbf{U}\blacksquare\mathbf{V}^T$. Mối liên hệ mật thiết giữa SVD và chéo hóa được tận dụng: các giá trị kỳ dị σ_i là căn bậc hai của các giá trị riêng của ma trận đối xứng $\mathbf{A}^T\mathbf{A}$.

3.2.2 Quy trình cài đặt

1. Tính $\mathbf{A}^T\mathbf{A}$.
2. Gọi hàm `diagonalize()` từ phần trước để tìm trị riêng và vector riêng của $\mathbf{A}^T\mathbf{A}$.
3. Chuẩn hóa các vector riêng để tạo ma trận trực giao $\mathbf{V}$.

4. Tính các vector kỳ dị trái $\mathbf{u}_i = \frac{1}{\sigma_i}\mathbf{A}\mathbf{v}_i$.
5. Sử dụng Gram-Schmidt để mở rộng $\mathbf{U}$ nếu ma trận không vuông ($m \neq n$).

3.3 Trực quan hóa biến đổi tuyến tính với Manim (manim_scene.py)

3.3.1 Ma trận thử nghiệm và ý nghĩa hình học

Để minh họa rõ nét nhất bản chất của phân rã SVD, nhóm sử dụng ma trận không vuông $A \in \mathbb{R}^{3\times 2}$ nhằm thể hiện sự biến đổi từ không gian hai chiều (2D) sang không gian ba chiều (3D):

$$A = \begin{bmatrix} 3 & 2 \\ 2 & 3 \\ 2 & -2 \end{bmatrix}$$

Vì ma trận A không phải là ma trận vuông, phương pháp chéo hóa trực tiếp $A = PDP^{-1}$ không thể áp dụng. Video animation do nhóm thực hiện giải thích cách SVD giải quyết vấn đề này bằng cách phân rã biến đổi phức tạp thành chuỗi ba bước hình học đơn giản:

$$\textbf{Rotate } (V^T) \rightarrow \textbf{Scale } (\Sigma) \rightarrow \textbf{Rotate } (U)$$

3.3.2 Cấu trúc kịch bản Animation

Video được nhóm xây dựng dựa trên kịch bản phân đoạn logic nhằm giúp người xem nắm bắt từ lý thuyết đến hình học trực quan:

- **Phân tích thành phần cốt lõi:** Giải thích vai trò của ma trận V (hướng quay 2D), Σ (mức độ co giãn $\sigma_i = \sqrt{\lambda_i}$) và U (hướng quay 3D).
- **Diễn họa hình học (Core Visualization):**
  1. **Khởi tạo:** Hiển thị lưới không gian và các vector cơ sở đơn vị trong $\mathbb{R}^2$.
  2. **Phép quay V^T:** Xoay hệ cơ sở theo các vector riêng của A^TA.
  3. **Phép co giãn Σ:** Biến đổi đường tròn đơn vị thành hình Ellipse 2D dựa trên các giá trị kỳ dị σ_1, σ_2.
  4. **Nâng không gian (2D $\rightarrow$ 3D):** Diễn họa việc đưa hình Ellipse vào không gian ba chiều $\mathbb{R}^3$.
  5. **Phép quay U:** Thực hiện phép quay cuối cùng để đưa các vector về đúng vị trí đích mà ma trận A quy định.

3.3.3 Kết quả thực nghiệm và Hiệu ứng

Thông qua việc sử dụng thư viện Manim, các khái niệm trừu tượng được cụ thể hóa bằng chuyển động. Việc quan sát hình Ellipse bị kéo giãn và xoay giúp người xem hiểu rằng SVD thực chất là việc tìm ra một hệ cơ sở mà ở đó phép biến đổi chỉ đơn thuần là co giãn.

Để kiểm chứng tính chính xác, nhóm đã thực hiện tính toán trên mã nguồn Python. Với ma trận mẫu A đã nêu, thuật toán cho kết quả sai số tái tạo cực nhỏ:

$$\varepsilon = \max |A - U\Sigma V^T| \approx 10^{-15} \tag{3}$$

Kết quả này khẳng định độ chính xác số học của thuật toán cài đặt đạt mức tối ưu, sẵn sàng cho các ứng dụng thực tế.

3.3.4 Ứng dụng thực tiễn và Thách thức kỹ thuật

Việc trực quan hóa không chỉ dừng lại ở lý thuyết mà còn mở rộng ra các ứng dụng thực tế mà nhóm đã tổng kết:

- **Nén dữ liệu:** Sử dụng xấp xỉ hạng thấp của SVD để giảm dung lượng ảnh mà vẫn giữ được đặc trưng chính.
- **Giảm chiều dữ liệu (PCA):** Tìm các phương chính trong Machine Learning.
- **Hệ thống gợi ý:** Phân tích ẩn dụ giữa người dùng và sản phẩm dựa trên các đặc trưng tiềm ẩn.

Khó khăn gặp phải: Trong quá trình thực hiện, nhóm đối mặt với việc đồng bộ các phép biến đổi trong không gian 3D của Manim đòi hỏi tính toán tọa độ chính xác tuyệt đối. Đồng thời, thời gian Render video 3D khá lâu, đặt ra yêu cầu phải tối ưu hóa mã nguồn Python để đạt hiệu suất cao hơn.

Tôi hiểu rồi, tôi sẽ tổng hợp lại toàn bộ các mảnh ghép để tạo thành một báo cáo Part 3 đầy đủ nhất, không bỏ sót bất kỳ lập luận học thuật hay đoạn mã nguồn nào.Cấu trúc này sẽ đi từ Cài đặt bộ giải (Solvers) → Giải thích toán học → Quy trình kiểm thử (Benchmark) → Phân tích ổn định số.Đoạn mã

4 PART 3: GIẢI HỆ PHƯƠNG TRÌNH TUYẾN TÍNH VÀ ĐÁNH GIÁ HIỆU NĂNG

4.1 Mục tiêu và Đặt vấn đề

Nhiệm vụ trọng tâm của Phần 3 là kết hợp các thuật toán đại số tuyến tính đã xây dựng để giải hệ phương trình $Ax = b$. Quá trình thực thi không chỉ dừng lại ở việc tìm ra nghiệm, mà còn cung cấp cơ chế phân tích tính ổn định số (*numerical stability*) thông qua 3 hướng tiếp cận cốt lõi: khử Gauss, phân rã ma trận suy rộng (SVD), và phương pháp lặp Gauss-Seidel.

4.2 Cài đặt thuật toán cốt lõi (solvers.py)

4.2.1 Khử Gauss với Partial Pivoting (`solve_gauss`)

Giải thuật dựa trên các biến đổi chuẩn để đưa hệ phương trình về dạng tam giác trên. Đặc biệt, ở đây áp dụng kỹ thuật **Partial Pivoting** (đổi chỗ dòng để đưa phần tử chéo có trị tuyệt đối lớn nhất lên trên). Kỹ thuật này triệt tiêu rủi ro chia cho một số quá nhỏ (gần 0), từ đó hạn chế tối đa sai số làm tròn khi thực thi trên máy tính (*Floating-point error*).

```
def solve_gauss(A, b):
    A_copy = [row[:] for row in A]
    b_copy = b[:]
    # Su dung ham gaussian_eliminate tu Part 1
    _, x, _ = gaussian_eliminate(A_copy, b_copy)
    if isinstance(x, str):
        raise ValueError(f"He phuong trinh vo nghiem: {x}")
    return x[0] if isinstance(x, tuple) else x
```

Listing 6: Triển khai giải hệ phương trình bằng khử Gauss

4.2.2 Phân rã ma trận SVD (`solve_svd`)

Sử dụng phân rã ma trận cực trị $A = U\Sigma V^T$. Phép giải dựa trên cấu trúc của ma trận nghịch đảo suy rộng phương pháp cực tiểu hóa bình phương (*Pseudoinverse*):

$$x = V\Sigma^{-1}U^T b \tag{4}$$

Xử lý ổn định số: Thuật toán đưa vào tham số `sv_threshold` (10^{-10}) áp dụng kỹ thuật *Truncated SVD*. Chỉ các giá trị kỳ dị (*singular values*) $\sigma_i >$ threshold mới được phép chia nghịch đảo. Thuật toán ngăn chặn triệt để hiện tượng phân kỳ khi hệ số ma trận cận suy biến.

```
def solve_svd(A, b, sv_threshold=1e-10):
    U, S, Vt = svd_decomposition(A)
    m, n = len(U), len(Vt)
    Ut_b = [sum(U[i][j] * b[i] for i in range(m)) for j in range(m)]
    sigma_inv_Ut_b = [0.0] * n
    for i in range(min(m, n, len(S))):
        if S[i] > sv_threshold:
            sigma_inv_Ut_b[i] = Ut_b[i] / S[i]
    x = [sum(Vt[j][i] * sigma_inv_Ut_b[j] for j in range(n)) for i in
   range(n)]
    return x
```

Listing 7: Giai bang SVD Pseudoinverse

4.2.3 Phương pháp lặp Gauss-Seidel (`gauss_seidel`)

Đây là lựa chọn hàng đầu cho các hệ thống ma trận vuông thưa (*sparse matrix*) kích thước rất lớn vốn không phù hợp với các phương pháp loại trừ trực tiếp.

Cơ sở toán học: Công thức lặp của Gauss-Seidel tính từng phần tử x_i ở vòng lặp thứ $k+1$ dựa trên cả các phần tử $x_j^{(k+1)}$ vừa được cập nhật ($j < i$) và các phần tử $x_j^{(k)}$ chưa được duyệt tới ở vòng lặp hiện tại ($j > i$):

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)}\right) \tag{5}$$

Sự tối ưu so với phương pháp Jacobi: Sự khác biệt mang tính bản lề so với Jacobi: Gauss-Seidel **sử dụng ngay lập tức các cập nhật x_j mới nhất** vừa được tính ở các hàng trước đó. Cơ chế lan truyền thông tin sớm này giúp tốc độ hội tụ nhanh gấp đôi. Trong hàm `gauss_seidel`, điều này được hiện thực hóa cực kì hiệu quả nhờ thao tác gán và ghi đè trực tiếp trên cùng một biến mảng (*in-place update*).

Điều kiện hội tụ mạnh mẽ và Tiêu chí dừng: Gauss-Seidel được chứng minh hội tụ hoàn toàn nếu cấu trúc ma trận A thỏa tính chất **chéo trội chặt hàng (*Strictly Diagonally Dominant*)**. Hệ thống liên tục đo lường mức độ biến thiên chuẩn vô cùng giữa hai kết quả liên tiếp:

$$\|x^{(k+1)} - x^{(k)}\|_\infty < \texttt{tol} = 10^{-10} \tag{6}$$

```
def gauss_seidel(A, b, x0=None, tol=1e-10, max_iter=1000):
    n = len(A)
    x = [0.0] * n if x0 is None else x0[:]
    for iteration in range(1, max_iter + 1):
        x_old = x[:]
        for i in range(n):
```

```
            sigma = sum(A[i][j] * x[j] for j in range(n) if j != i)
            x[i] = (b[i] - sigma) / A[i][i]
        if max(abs(x[i] - x_old[i]) for i in range(n)) < tol:
            return x, iteration, True
    return x, max_iter, False
```

Listing 8: Trien khai Gauss-Seidel in-place update

4.3 Quy trình thực nghiệm (benchmark.py)

Để đánh giá khách quan, nhóm xây dựng chương trình benchmark tự động trên 3 loại cấu trúc: ma trận ngẫu nhiên chéo trội, ma trận SPD (*well-conditioned*) và ma trận Hilbert (*ill-conditioned*). Hệ thống sử dụng hai thang đo cốt lõi:

- **Số điều kiện (Condition Number):** Xác định qua tỷ số giá trị kỳ dị lớn nhất và nhỏ nhất $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$ để đo lường độ ổn định dựa trên chuẩn phổ (*Spectral norm*).
- **Sai số tương đối (Relative Error):** Sử dụng đánh giá phần dư để tính toán sai số độc lập với nghiệm lý thuyết:

$$\text{Error} = \frac{\|A\hat{x} - b\|_2}{\|b\|_2} \tag{7}$$

4.4 Phân tích kết quả thực nghiệm

4.4.1 Hiệu năng và Độ phức tạp thời gian (Time Complexity)

Dựa trên dữ liệu thực nghiệm, nhóm thực hiện vẽ đồ thị trên hệ tọa độ Log-Log để so sánh với đường $O(n^3)$ lý thuyết.

```
import pandas as pd
import seaborn as sns

# Ve duong O(n^3) ly thuyet de so sanh
n_values = df_random['n'].unique()
t_theory = [ (n**3) * (df_random['avg_time'].iloc[0]/(df_random['n'].iloc
    [0]**3)) for n in n_values]

sns.lineplot(data=df_random, x='n', y='avg_time', hue='name', marker='o')
plt.plot(n_values, t_theory, '--', label='Ly thuyet O(n^3)', color='black
    ')
plt.xscale('log')
plt.yscale('log')
```

Listing 9: Xử lý dữ liệu và vẽ đồ thị Log-Log bằng Seaborn

Nhận xét:

- **Khử Gauss và SVD:** Đường biểu diễn thực nghiệm có độ dốc xấp xỉ 3, khớp với lý thuyết $O(n^3)$. Trong đó, SVD có chi phí cao hơn do phải xử lý các ma trận trực giao.
- **Gauss-Seidel:** Thể hiện ưu thế vượt trội về tốc độ và bộ nhớ trên các ma trận chéo trội nhờ cơ chế cập nhật trực tiếp (*in-place update*). Phương pháp này thường hội tụ sau ít hơn 15 vòng lặp, tuy nhiên hiệu suất giảm mạnh nếu ma trận không thỏa điều kiện hội tụ.

4.4.2 Tính ổn định số và Số điều kiện (Numerical Stability)

Sự tương quan giữa cấu trúc ma trận và sai số được thể hiện rõ qua kịch bản kiểm thử giới hạn giữa ma trận SPD (ổn định) và Hilbert (điều kiện xấu).

- **Ma trận SPD:** Có số điều kiện $\kappa(A) \approx 1$. Sai số tương đối duy trì ổn định ở mức $10^{-15} \sim 10^{-16}$, tương đương sai số máy (*Machine Epsilon*).
- **Ma trận Hilbert:** Khi kích thước n tăng, $\kappa(H_n)$ tăng vọt theo hàm mũ. Theo lý thuyết sai số:
$$\frac{\|\hat{x} - x\|}{\|x\|} \leq \kappa(A) \cdot \frac{\|\delta b\|}{\|b\|} \tag{8}$$
Do $\kappa(A)$ quá lớn, các sai số làm tròn nhỏ nhất bị phóng đại làm nghiệm của phương pháp Gauss bị sai lệch nặng nề. Tại đây, **Truncated SVD** thể hiện ưu thế vượt trội bằng cách loại bỏ các trị kỳ dị $< 10^{-10}$, giúp ngăn chặn hiện tượng phân kỳ nghiệm.

4.5 Tổng kết và Kiến nghị

Từ kết quả thực nghiệm, nhóm đưa ra kiến nghị lựa chọn thuật toán:

Đặc điểm hệ thống	**Thuật toán tối ưu**	**Lý do**
Hệ đặc, ổn định	Gauss (Partial Pivot)	Cân bằng tốc độ và độ chính xác
Hệ điều kiện kém	Phân rã SVD	Xử lý tốt các ma trận gần suy biến
Hệ lớn, ma trận thưa	Gauss-Seidel	Tiết kiệm bộ nhớ, hội tụ nhanh

Bảng 1: Bảng tổng kết lựa chọn phương pháp giải hệ phương trình tuyến tính.

5 KẾT LUẬN

- **Về kết quả đạt được:** Nhóm đã xây dựng hoàn thiện hệ thống thư viện tính toán đại số tuyến tính có độ chính xác cao. Hệ thống không chỉ xử lý tốt các bài toán cơ bản mà còn giải quyết triệt để các trường hợp đặc biệt như ma trận cận suy biến, hệ phương trình vô số nghiệm (xác định không gian nghiệm tổng quát) và các ma trận có số điều kiện lớn. Điều này có được nhờ vào việc tích hợp linh hoạt phân rã SVD, phương pháp lặp Gauss-Seidel và kiểu dữ liệu số hữu tỷ (`fractions.Fraction`). Kết quả thực nghiệm cho thấy sai số tái tạo nghiệm đạt mức tối ưu (10^{-15}), tiệm cận sai số máy (*Machine Epsilon*).
- **Bài học kinh nghiệm:** Đồ án giúp nhóm hiểu sâu sắc mối liên hệ giữa lý thuyết toán học thuần túy và tính toán thực tiễn trên máy tính. Nhóm đã nhận thức rõ tầm quan trọng cốt lõi của kỹ thuật chọn phần tử trội (*Partial Pivoting*) và ngưỡng lọc trị kỳ dị (*Truncated SVD*) trong việc kiểm soát tính ổn định số học (*Numerical Stability*).
- **Thách thức và Hạn chế:** Quá trình thực thi đòi hỏi tư duy logic khắt khe, đặc biệt là việc quản trị chỉ số ẩn tự do trong thuật toán thế ngược và đồng bộ hóa các phép biến đổi hình học trong không gian đa chiều khi trực quan hóa bằng Manim.

6 TÀI LIỆU THAM KHẢO

1. *Đồ án 1: Ma trận và Tính toán Khoa học, Khoa CNTT - Trường ĐH Khoa học Tự nhiên, ĐHQG-HCM.*

2. *Chương 1: System of Linear Equations*, Bài giảng môn Toán ứng dụng và Thống kê, Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên - ĐHQG HCM, 2026.
3. *Chương 2: Matrix Decomposition*, Bài giảng môn Toán ứng dụng và Thống kê, Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên - ĐHQG HCM, 2026.
4. *Chương 4: Diagonalization*, Bài giảng môn Toán ứng dụng và Thống kê, Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên - ĐHQG HCM, 2026.
5. *Chương 5: Singular Value Decomposition (SVD)*, Bài giảng môn Toán ứng dụng và Thống kê, Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên - ĐHQG HCM, 2026.
6. GeeksforGeeks, *Gauss–Seidel method in Python*, https://www.geeksforgeeks.org/python-gauss-seidel-method/
7. Vũ Hữu Tiệp, *Singular Value Decomposition*, Machine Learning cơ bản, https://machinelearningcoban.com/2017/06/07/svd/, [Truy cập ngày April 17, 2026].
8. Phạm Đình Khánh, *Phân tích thành phần chính (PCA)*, Deep AI Book, https://phamdinhkhanh.github.io/deepai-book/ch_ml/PCA.html, [Truy cập ngày April 17, 2026].

7 BẢNG PHÂN CÔNG CÔNG VIỆC

Thành viên	MSSV	Nội dung công việc phụ trách
Nguyễn Khánh Đăng	24120171	• Part 1: `determinant.py`, `gaussian.py` • Part 3: `analysis.ipynb` • Viết và tổng hợp báo cáo.
Đặng Quang Huy	24120322	• Part 2: diagonalization.py • Part 3: `solvers.py` • Viết báo cáo.
Trương Bảo Nguyên	24120397	• Part 2: decomposition.py • Part 3: `benchmark.py` • Viết báo cáo.
Võ Gia Phúc	24120413	• Part 2: `demo_video.pdf`, `manim_scene.py` • Viết báo cáo.
Trần Nguyễn Lâm Gia Thụ	24120458	• Part 1: `inverse.py`, `rank_and_basic.py`, `part1_demo.ipynb` • Viết báo cáo.